Week 5 Reading: ReLU Graphs and MILP Derivations

Optimization formulation for reactor design

At a fixed flow of 1 L/min, find the minimum volume satisfying the product specifications. Since $V = Q\tau$, residence time τ expresses the design choice.

The fixed context is 330 K and an A feed concentration of 1.2 mol/L. Outputs are A, B, and C concentrations; require $B \geq 0.40$ and $C \leq 0.25$ mol/L.

We first define the process problem, translate the predictor into constraints, and then verify the computed solution.

Feasible set of the CSTR design problem

$$\min_{\tau,\, y} V = Q\tau$$
$$2 \leq \tau \leq 10, y = f_\theta\left(\tau\right)$$
$$y_B \geq .40, y_C \leq .25, y \geq 0$$

The τ range defines both the model domain and the starting point for bound calculation. Concentration inequalities encode product requirements, while y = fθ(τ) encodes the predictor.

The objective V is in L; y is in mol/L. τ and V have the same numerical value only because Q = 1 L/min.

Interpolation data and model construction

τ [min]	A [mol/L]	B [mol/L]	C [mol/L]
2	0.857143	0.285714	0.057143
4	0.666667	0.380952	0.152381
6	0.545455	0.409091	0.245455
8	0.461538	0.410256	0.328205
10	0.400000	0.400000	0.400000

Interpolate linearly between adjacent rows. Each segment is a convex combination of its endpoint concentrations, preserving nonnegativity and the total 1.2 within the domain.

The table displays six decimals; computations use full-precision coefficients.

Derivation of a ReLU representation of linear interpolation

Let c_i be the concentration vector at knot i and compute each segment slope s_i. Start with the first slope and add its change at each breakpoint.

$$s_i = \frac{c_{i+1} - c_i}{\tau_{i+1} - \tau_i}$$

$$y(\tau) = c_0 + s_0(\tau - \tau_0) + \sum_{i=1}^{3} (s_i - s_{i-1}) \max(0, \tau - \tau_i)$$

Within segment i, active hinge slopes telescope to $s_0 + (s_1 - s_0) + \cdots + (s_i - s_{i-1}) = s_i$. Continuity at the knots and the correct initial value identify the desired interpolant.

All weights after input scaling

Use $x = (\tau - 2)/8$, $h_j = \max(0, x - t_j)$, and $t = [0, 0.25, 0.50, 0.75]$. In $y = b + W^T h$, rows of W index neurons and columns index components.

j	W_jA	W_jB	W_jC
0	-0.761904762	0.380952381	0.380952381
1	0.277056277	-0.268398268	-0.008658009
2	0.149184149	-0.107892108	-0.041292041
3	0.089510490	-0.045687646	-0.043822844

b = [0.857142857, 0.285714286, 0.057142857] mol/L. W is also in mol/L, while h is dimensionless.

Full-precision coefficients are stored in `frozen_network.json`. Reconstructing them from the rounded table can introduce small output differences.

Both directions of Big-M equivalence

$$h \geq 0, h \geq a$$
$$h \leq a - L\,(\,1 - z\,)$$
$$h \leq Uz, z \in \{\,0, 1\,\}$$

A feasible formulation point with z = 0 has h = 0 and a ≤ 0; with z = 1 it has h = a ≥ 0. Either branch gives h = max(0, a).

Conversely, choose any point on the ReLU graph. Set z = 0 if a ≤ 0 and z = 1 if a ≥ 0; all four inequalities hold. Valid bounds L ≤ a ≤ U are needed in this direction too.

Equivalence of multilayer ReLU formulations

Fix input x. The first affine layer uniquely determines a^1, and its ReLU constraints determine h^1 exactly.

Once h^1 is fixed, the next affine layer determines a^2. Repeating this argument through the final layer shows that the MILP output equals the forward output.

Conversely, insert all forward intermediate values and choose each z from its neuron’s sign to obtain an MILP-feasible point. Bounds must be valid at every layer; nonunique z values at zero do not change the output.

Layerwise propagation of interval bounds

$$a = \sum_i w_i h_i + b$$
$$L = b + \sum_i \min\left(w_i l_i, w_i u_i \right)$$
$$U = b + \sum_i \max\left(w_i l_i, w_i u_i \right)$$

After ReLU, the interval is [max(0, L), max(0, U)]. Pass it to the next affine layer as its input bounds.

For example, if $h_1 = x$ and $h_2 = x$, then $h_1 - h_2$ is always zero. Using only the separate intervals [0, 1] gives [−1, 1]. This remains valid but loses their correlation.

Variable ordering in the objective and constraint matrix

The variable order is [x, h_0, h_1, h_2, h_3, y_A, y_B, y_C, z_1, z_2, z_3]. Each matrix row is stored as row_lower ≤ Av ≤ row_upper.

For V = 2 + 8x, the constant 2 does not change the optimizer, so the solver minimizes 8x. Add 2 back when reporting V and its dual bound.

An integer variable becomes binary only when its bounds are [0, 1]. If pre-activations are explicit variables, assign lower bounds that permit negative values.

Compute the optimal residence time by hand

B stays below 0.40 throughout 2-4 min. It increases over 4-6 min, so compute the first point reaching 0.40.

$$\tau = 4 + 2\frac{.40 - .380952380952}{.409090909091 - .380952380952}$$
$$\tau \approx 5.353846\text{min}$$

The same interpolation fraction gives [A, B, C] = [0.584615, 0.400000, 0.215385] mol/L. This satisfies C ≤ 0.25 and nonnegativity, so it is the minimum feasible residence time.

The continuous relaxation of a scalar ReLU

For L < 0 < U, relax z to [0, 1] and eliminate it to obtain a triangle.

$$h \geq 0, h \geq a$$
$$h \leq \frac{U}{U-L}\left(\, a - L \,\right)$$

The upper inequalities give h/U ≤ z ≤ (h − a + L)/L. Combining them, taking care with the negative sign of L, yields the last inequality.

This is the convex hull of the scalar (a, h) graph. When a is an affine function of several inputs, it need not be the strongest relaxation in the original input space.

MILP representation of affine equality projection

$$y = Py_{\text{raw}} + Ru + q$$

If the Week 4 projection has this affine form with fixed P, R, and q, append it directly as output equalities. No additional binaries are required.

If the constraint matrices depend on the decisions, inverses or products of variables may appear, so the same conclusion does not automatically apply.

When adding a separate conservation equality, check compatibility with existing outputs. A new equality imposed on an approximate predictor can shrink or empty its feasible set.

Exercise 1: signs and branches

For $a = 2x - 1$ and $x \in [0, 1]$, find L and U and write the Big-M inequalities. Calculate (a, h, z) at $x = 0.2$ and 0.8.

Check: $L = -1$, $U = 1$, with $h \geq 0$, $h \geq 2x - 1$, $h \leq 2x - z$, and $h \leq z$.

At $x = 0.2$ the tuple is $(-0.6, 0, 0)$; at $x = 0.8$ it is $(0.6, 0.6, 1)$. At $x = 0.5$, $h = 0$ and either binary value is possible.

Exercise 2: product specifications and feasibility

Does feasibility remain if B ≥ 0.41 replaces B ≥ 0.40 while C ≤ 0.25 stays unchanged? Use the knot table and segmentwise interpolation.

Check: C is 0.245455 at 6 min and 0.328205 at 8 min. The C bound limits residence time to about 6.109859 min.

B there is about 0.409155 mol/L, and its values in earlier segments are smaller. The modified surrogate problem is therefore infeasible. This calculation helps distinguish an infeasible solver result from an implementation error.

Exercise 3: inputs for formulation verification

Random checks can miss errors at breakpoints or domain endpoints. Include $\tau = 2, 4, 6, 8, 10$ and points on both sides of interior knots.

At each fixed input, measure the maximum absolute difference in h and y. At $a = 0$, require output agreement rather than identical binary values.

Check: the supplied run had a maximum difference of about 1.25×10^{-8} across 41 inputs. Solver tolerances can leave small differences very close to a breakpoint. Distinguish mathematical equivalence of a valid formulation from finite-precision computation.

Lab files and sources

`week05_lab.py` includes coefficient construction, MILP matrix assembly, fixed-input checks, and independent segment calculations. Use `frozen_network.json` and `fit_knots.csv` to inspect the model; use `results.json` and `milp_matrix.csv` to inspect the solve.

Boyd & Vandenberghe, Convex Optimization, §4.1: variables, objectives, and feasible sets.

Anderson et al., Strong mixed-integer programming formulations for trained neural networks, §2.1: the ReLU Big-M formulation.

SciPy milp documentation: bounds, integrality, LinearConstraint, and return status.

Lecture · Lab bundle